SRS For Building Materials Management System (BMMS) for Deshmukh Traders

Version 1.0

Prepared by Team

STRUCTECH

Parimal Dode

Varad Kengale

Rajdeep Avhad

Harshada Kolekar

Sakshi Deshmukh

Dnyaneshwar Kote

Client

Deshmukh Traders, Jalgaon

Department of Computer Engineering

MITAOE

Table of Contents

1. INTRODUCTION.....4

1.1 Purpose.....4
1.2 Intended Audience and Reading Suggestions.....4
1.3 Project Scope.....4
1.4 References.....5
1.5 SRS Structure Overview.....5

2. OVERALL DESCRIPTION.....6

2.1 Product Perspective.....6
2.2 Product Function.....6
2.3 User / Stakeholder Classes and Characteristics.....7
2.4 Operating Environment and Hardware Descriptions.....7
2.5 Design and Implementation Constraints.....8
2.6 Business Requirements.....8
2.6.1 Business Opportunity.....8
2.6.2 Business Objectives and Success Criteria.....9
2.6.3 Customer or Market Needs.....9
2.6.4 Assumptions and Dependencies.....9

3. SYSTEM FEATURES.....10

Module A: Authentication & User Management.....10
3.1. Description.....10
3.2 Functional Requirements.....10
Module B: Product and Price Management.....10
3.1. Description.....10
3.2 Functional Requirements.....10
Module C: Inventory and Stock Management.....10
3.1. Description.....10
3.2 Functional Requirements.....11
Module D: Order and Billing Management.....11
3.1. Description.....11
3.2 Functional Requirements.....11
Module E: Reporting and Notifications.....11
3.1. Description.....11
3.2 Functional Requirements.....11

4. NONFUNTIONAL REQUIREMENTS.....12

4.1 Performance Requirements.....12
4.2 Reliability Requirements.....12
4.3 Security requirements.....12
4.4 External Interface Requirements.....13
4.4.1 User Interfaces.....13
4.4.2 Hardware Interface.....13

APPENDIX A: Glossary..... 14

APPENDIX B: Diagrams..... 15

APPENDIX C: Algorithm Design..... 30

APPENDIX D: Implementation plan..... 31

APPENDIX E: Testing Strategy.....32

1 INTRODUCTION

1.1 Purpose

The purpose of this document is to specify the software requirements for the **Building Materials Management System (BMMS)**. This document is intended for the developers, the client (**Deshmukh Traders**), and the testing team. It describes the functional and non-functional requirements, system interfaces, and design constraints for the proposed system.

1.2 Problem Statement

The current system used by building material stores manages product pricing, orders, and inventory manually, which can lead to inefficiency and errors in daily operations.

1. **Price and Cost Calculation Issues:** Prices of materials like cement and steel change frequently, and calculating the final cost including transport, labor, loading, and GST manually may lead to incorrect pricing.
2. **Inventory Management Problems:** Stock records are maintained manually, which may cause inaccurate inventory tracking and no automatic alerts when stock levels become low.
3. **Order Management Issues:**
   Orders are recorded manually, which may lead to mistakes in order quantity, delayed processing, and difficulty in tracking customer orders.

1.3 Project Scope

The **Building Materials Management System (BMMS)** is a web-based application designed to help building material stores manage products, pricing, orders, and inventory efficiently. The system will display materials such as cement, steel, water tanks, and accessories, fetch daily market prices for cement and steel, and automatically calculate the final cost by including transportation, labor, loading charges, GST, and bulk discounts.

The system mainly focuses on:

- Displaying building materials such as Cement, Steel, Water Tanks, and Accessories.
- Fetching daily market prices for cement and steel using web scraping.

- Calculating the final cost including transportation, labor, loading charges, and GST.
- Applying bulk discounts automatically for large orders.
- Managing product inventory and stock levels.
- Sending alerts to the owner when stock goes below a certain limit.

The system will mainly be used by two types of users:

Customer

- View products and materials
- Place orders or request quotes
- Check calculated final price

Store Owner (Admin)

- Manage products and stock
- Monitor orders
- Receive stock alerts
- Update product information

1.4 Definitions, Acronyms, and Abbreviations

BMMS – Building Materials Management System
GST – Goods and Services Tax (18%)
Admin – Store owner who manages products and stock
Inventory – The quantity of materials available in stock
Dynamic Pricing – Automatic updating of product prices based on market changes

1.5 References

- IEEE Std 830-1998, IEEE Recommended Practice for Software Requirements Specifications
- Software Engineering: A Practitioner's Approach – R. S. Pressman
- Online market platforms such as IndiaMart for market price reference

2. OVERALL DESCRIPTION

2.1 Product Perspective

The **Building Material Store Management System (BMSMS)** is a new digital solution designed to replace the existing manual record-keeping system used in building material shops. Currently, store owners maintain product pricing, inventory, and customer orders using paper registers or basic spreadsheets, which often leads to errors and inefficiency.

The proposed system will provide a web-based platform where store owners can manage products, update daily prices of materials such as cement and steel, calculate the final cost including transport, labor, loading charges, and GST, and monitor stock levels.

The system will operate in a cloud-based environment with a centralized database, enabling real-time updates and better management of store operations.

2.2 Product Functions

The major functions of the system include:

Price Management

- Update daily prices of building materials such as cement and steel.
- Automatically calculate the final price including transportation, labor, loading charges, and GST.

Inventory Management

- Maintain records of available stock for all products.
- Automatically notify the owner when stock levels become low.

Order Management

- Allow recording and tracking of customer orders.
- Calculate total order cost automatically.

Product Management

- Add, update, or remove building materials from the system.
- Maintain product details such as name, price, quantity, and category.

Reporting and Analytics

- Generate reports for sales, stock levels, and daily transactions.
- Help store owners make better business decisions.

2.3 User / Stakeholder Classes and Characteristics

Store Owner (Admin)

- Primary user of the system.
- May have basic computer knowledge but not advanced technical skills.
- Requires a simple and easy-to-use interface for managing inventory, prices, and orders.

Store Staff / Employee

- Responsible for entering product data, updating stock, and managing orders.
- Requires a quick and efficient interface for daily operations.

Customers

- End users who purchase building materials.
- Interested in accurate pricing and quick order processing.

2.4 Operating Environment and Hardware Descriptions

The system will operate in the following environment:

Frontend

- Web-based user interface developed using HTML, CSS, and JavaScript (or React.js).

Backend

- Server-side application developed using Node.js / PHP / Python.

Database

- MySQL or PostgreSQL database to store product, inventory, and order data.

Hardware Requirements

- Computer or laptop for store owner.

- Internet connection for accessing the system.
- Modern web browser such as Chrome, Edge, or Firefox.

Operating System

- Windows, Linux, or MacOS.

2.5 Design and Implementation Constraints

Price Accuracy

- The system must ensure accurate calculation of product prices including GST and additional charges.

Security

- User authentication must be implemented to prevent unauthorized access.

Performance

- The system should provide quick response times for inventory and price updates.

Data Integrity

- All product and order data must be stored securely and consistently.

Usability

- The interface should be simple so that non-technical users can operate it easily.

2.6 Business Requirements

2.6.1 Business Opportunity

Many building material stores still rely on **manual record keeping**, which leads to inefficiency, pricing errors, and poor inventory tracking.
Developing a **digital management system** can improve store operations, reduce errors, and increase productivity.

2.6.2 Business Objectives and Success Criteria

Business Objectives

- Improve **inventory management efficiency**.
- Reduce **manual calculation errors**.
- Provide **accurate price calculation including GST and transport costs**.
- Improve overall **store management and decision making**.

Success Criteria

- Reduction in pricing errors.
- Faster order processing.
- Improved stock management and reduced product shortages.

2.6.3 Customer or Market Needs

Customers and store owners require:

- **Accurate and transparent pricing** for building materials.
- Quick **order processing and billing**.
- Easy access to **product availability and stock information**.
- A reliable system to **manage inventory and reduce manual work**.

2.6.4 Assumptions and Dependencies

- It is assumed that the **store owner has access to a computer or smartphone**.
- The system depends on **stable internet connectivity** for cloud-based operations.
- Product prices and GST rates will be **updated regularly by the store owner**.

3. SYSTEM FEATURES

Module A: Authentication & User Management

Description

This module manages **secure access to the system** for admins and customers. It ensures that only authorized users can log in and use the system.

Functional Requirements

User Registration: Users can create a new account.

Secure Login: Users login using username/email and password.

Role-Based Access: Different access for **Admin and Customer**.

Profile Management: Users can update their personal details.

Secure Logout: Users can logout safely from the system.

Module B: Product and Price Management

Description

This module manages **building materials such as cement, steel, water tanks, and accessories**, along with their prices.

Functional Requirements

Add Products: Admin can add new products.

Update Products: Admin can edit product details and prices.

Delete Products: Admin can remove products from the system.

Price Updates: System updates daily market prices.

Product View: Customers can view available materials and prices.

Module C: Inventory and Stock Management

Description

This module manages the stock levels of building materials and helps track inventory.

Functional Requirements

Stock Tracking: System stores available quantity of each product.

Stock Update: Admin can update stock levels.

Automatic Stock Reduction: Stock decreases when orders are placed.

Low Stock Alert: Admin receives alerts when stock is low.

Module D: Order and Billing Management

Description

This module handles **customer orders and billing calculations** including additional charges and discounts.

Functional Requirements

Order Placement: Customers can select materials and place orders.

Cost Calculation: System calculates total material cost.

Additional Charges: Transport, labor, and GST are added automatically.

Bulk Discount: Discount applied for large orders.

Invoice Generation: System generates order bills.

Module E: Reporting and Notifications

Description

This module provides **reports and alerts** to help the admin monitor sales and inventory.

Functional Requirements

Sales Reports: System generates sales reports.

Inventory Reports: Shows stock details of materials.

Low Stock Notification: Alerts when stock is low.

Order History: Displays previous customer orders.

4. NON-FUNCTIONAL REQUIREMENTS

4.1 Performance Requirements

The system must respond quickly to ensure smooth user experience while viewing products, checking prices, and placing orders.

- Product and price information should load within **2 seconds**.
- Cost calculation and bill generation should take **less than 1 second**.
- The system should support **100+ concurrent users** without performance issues.
- Web scraping for price updates should run **automatically once per day**.

4.2 Reliability Requirements

The system must operate reliably to ensure continuous availability of product and price information.

- The system should maintain **99% uptime** during operational hours.
- Data must be **automatically saved in the database** to prevent data loss.
- In case of server failure, the system should **restore data using backups**.
- Inventory and billing data should remain **consistent and accurate**.

4.3 Security Requirements

The system must protect user data and prevent unauthorized access.

- User authentication should be protected using **secure login and encrypted passwords**.
- APIs should be protected using **secure authentication mechanisms (JWT or token-based security)**.
- Only **Admin users** can modify product prices and inventory.
- Sensitive data such as **user credentials must not be exposed**.

4.4 EXTERNAL INTERFACE REQUIREMENTS

4.4.1 User Interfaces

The system provides simple interfaces for both **Admin and Customers**.

Admin Dashboard:
Web-based interface to manage products, update prices, monitor stock, and view reports.

Customer Interface:
User-friendly interface where customers can **view materials, check prices, and place orders**.

Product Display Page:
Shows building materials such as **cement, steel, water tanks, and accessories with updated prices**.

4.4.2 Hardware Interface

The system interacts with basic hardware devices used to access the application.

- The system runs on **computers, laptops, or mobile devices**.
- Requires a **stable internet connection** for price updates and data synchronization.
- The server hosting the system must have **sufficient storage for product and order data**.
- Users access the system through **standard web browsers like Chrome, Edge, or Firefox**.

APPENDIX A: GLOSSARY

BMMS: Building Materials Management System.

Admin: User who manages products, prices, stock, and reports.

Customer: User who views materials and places orders.

Web Scraping: Automatic method to collect daily market prices from websites.

Inventory: Record of available building material stock.

Bulk Discount: Discount applied for large quantity orders.

GST: Government tax added to the final bill.

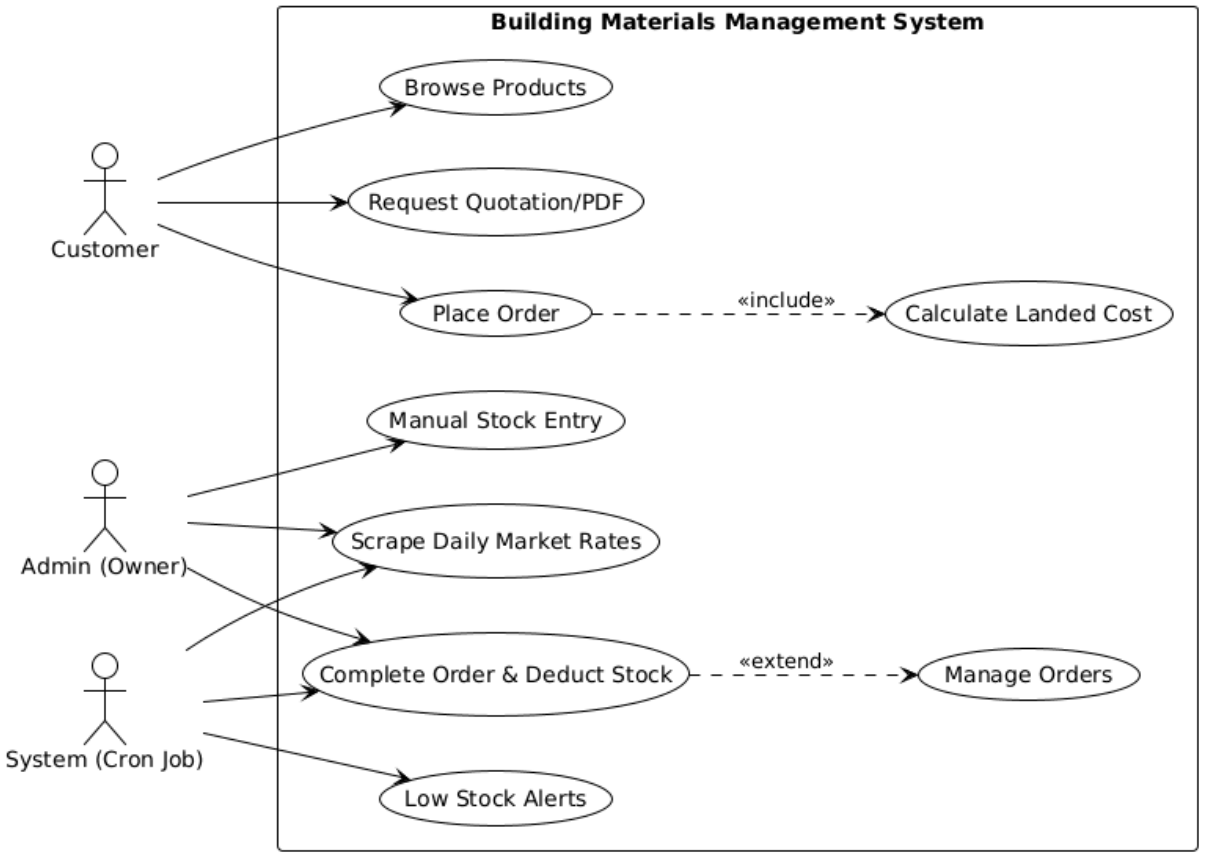
Invoice: Bill containing product details, quantity, and total cost.

Database: System used to store product, user, and order data.

API: Interface that allows communication between different system components

APPENDIX B: Diagrams

1. UML Overview Diagram



Field	Description
USE-CASE:1	Place Order & Calculate Landed Cost
PRIMARY ACTOR:	Customer
GOAL IN CONTEXT:	To allow a customer to select building materials, receive an accurate real-time quote (landed cost), and finalize a purchase order.
PRECONDITIONS:	1) System is online. 2) PostgreSQL database is populated with product IDs. 3) Scraper Service is functional to verify daily rates.
TRIGGER:	Customer selects material and quantity and requests a quote.
SCENARIO:	1) Customer browses materials and inputs quantity and delivery details. 2) System checks if current market prices are outdated (> 24hrs). 3) If outdated: System triggers Scraper Service to fetch and update rates. 4) Pricing Engine calculates landed cost (Base + Loading + Labor + Transport + GST). 5) System displays the itemized quote to the customer. 6) Customer confirms and clicks "Place Order". 7) System saves the order record and auto-deducts stock from inventory.

Field	Description
USE-CASE:1	Place Order & Calculate Landed Cost
PRIMARY ACTOR:	Customer
GOAL IN CONTEXT:	To allow a customer to select building materials, receive an accurate real-time quote (landed cost), and finalize a purchase order.
PRECONDITIONS:	1) System is online. 2) PostgreSQL database is populated with product IDs. 3) Scraper Service is functional to verify daily rates.
EXCEPTIONS:	1) If the Scraper Service cannot reach external sites, the system uses the last cached price and flags a warning to the Admin. 2) If the stock quantity is insufficient for the order, the system prevents placement and notifies the user. 3) If the user cancels at the quote stage, the process terminates without saving a record.
PRIORITY:	Essential, must be implemented.
WHEN AVAILABLE:	First increment (MVP).
FREQUENCY OF USE:	High (Continuous throughout the day).
CHANNEL TO ACTOR:	Via Web/Mobile Storefront (React/PWA).
SECONDARY ACTOR:	System (Cron Job/Scraper), Admin (for order review).
CHANNELS TO SECONDARY ACTORS:	Automated API triggers and Admin Dashboard notifications.
OPEN ISSUES:	1) What is the fallback if the external pricing site (IndiaMart/BankBazaar) is down? 2) Should the bulk discount (qty > 50) be adjustable by the Admin dynamically?

Field	Description
USE-CASE:2	Update Market Rates via Scraper Service
PRIMARY ACTOR:	System (Automated Scraper)
GOAL IN CONTEXT:	To automatically fetch the latest market prices for materials (Steel, Cement, Sand) from external sources to ensure the pricing engine remains accurate.
PRECONDITIONS:	1) Internet connectivity is active. 2) Target URL (e.g., IndiaMart/BankBazaar) is accessible. 3) Scraper module is initialized.
TRIGGER:	1) A scheduled time (e.g., every day at 8:00 AM). 2) A manual override by the Admin. 3) A "Place Order" request for a product whose price hasn't been updated in 24 hours.
SCENARIO:	1) The System identifies the list of materials needing a price update. 2) The Scraper Service navigates to the pre-defined external pricing URLs. 3) The Service extracts the raw price data using the scraping script. 4) The System cleans the data (removes symbols, converts units). 5) The System updates the productPrice attribute in the PostgreSQL database.

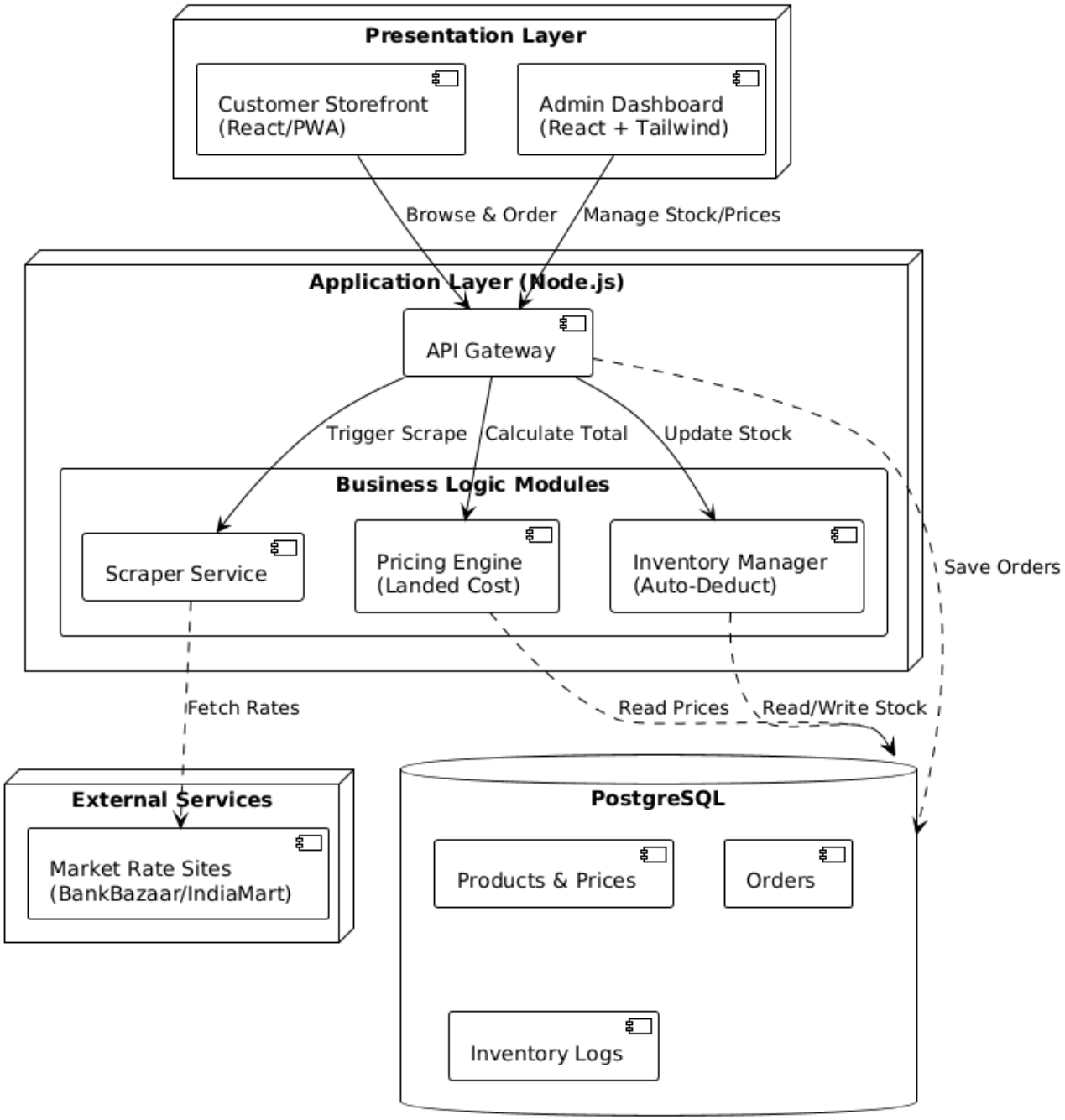
	6) The System logs the timestamp of the successful update.
EXCEPTIONS:	1) Source Site Down: System retries 3 times; if failure persists, it sends a "Critical Failure" alert to the Admin and uses the cached price. 2) Format Change: If the external site changes its HTML structure, the scraper fails to find the price; the Admin is notified to update the scraper logic.
PRIORITY:	High (Essential for profit margin protection).
WHEN AVAILABLE:	First increment (MVP).
FREQUENCY OF USE:	Daily (Scheduled) or on-demand.
CHANNEL TO ACTOR:	Internal API calls / Headless Browser.
SECONDARY ACTOR:	Admin (receives notifications of success/failure).
CHANNELS TO SECONDARY ACTORS:	Admin Dashboard / Email Notification.
OPEN ISSUES:	1) How to handle price discrepancies between different external sources? 2) Should the system store a history of prices for trend analysis?

Field	Description
USE-CASE:	Inventory Adjustment & Low Stock Alert
PRIMARY ACTOR:	Admin / Warehouse Manager
GOAL IN CONTEXT:	To update stock levels after a sale or physical delivery and trigger alerts when materials fall below a safe threshold.
PRECONDITIONS:	1) Admin is authenticated. 2) Product ID exists in the database. 3) stockQuantity attribute is initialized.
TRIGGER:	1) An order is successfully placed (Auto-deduct). 2) New stock arrives at the warehouse (Manual add). 3) A periodic system check of stock levels.
SCENARIO:	1) Admin logs into the Dashboard and selects "Manage Inventory." 2) Admin inputs the quantity received for a specific Material ID (e.g., Cement Bags). 3) System updates the stockQuantity in the database. 4) System compares the new stockQuantity against the minimumThreshold (e.g., 50 units). 5) If stock < threshold: System highlights the item in RED on the dashboard and sends a "Low Stock" notification. 6) Admin confirms the update and logs out.
EXCEPTIONS:	1) Negative Inventory: System prevents deduction if the order exceeds current stock, unless "Backorder" is enabled. 2) Database Lock: If two admins update the same item simultaneously, the system uses optimistic locking to prevent data corruption.

PRIORITY:	High (Prevents overselling and project delays).
WHEN AVAILABLE:	Second increment.
FREQUENCY OF USE:	Daily (Every time a truck leaves or arrives).
CHANNEL TO ACTOR:	Admin Web Portal / Dashboard.
SECONDARY ACTOR:	PostgreSQL Database.
CHANNELS TO SECONDARY ACTORS:	SQL Update Queries.
OPEN ISSUES:	1) Should the system automatically generate a "Purchase Order" to suppliers when stock is low? 2) Do we need to track "Batch Numbers" for expiring materials like Cement?

Field	Description
USE-CASE:	Admin Dashboard & Sales Analytics
PRIMARY ACTOR:	Admin
GOAL IN CONTEXT:	To provide a high-level overview of sales trends, revenue, and material price fluctuations over time.
PRECONDITIONS:	1) Admin is logged in. 2) Database contains historical order data and price logs. 3) Analytics engine is active.
TRIGGER:	Admin navigates to the "Reports" or "Dashboard" tab.
SCENARIO:	1) System queries the PostgreSQL database for total sales in the last 30 days. 2) System fetches the PriceHistory log to show how market rates (Scraper data) have trended. 3) System generates a visual chart showing "Top Selling Materials" (e.g., TMT Bars vs. Cement). 4) Admin filters the data by date range or material category. 5) System displays the "Profit Margin" by comparing Scraped Cost vs. Selling Price.
EXCEPTIONS:	1) No Data: If the database is empty (new install), the system displays "No records found" instead of an error. 2) Slow Query: If the dataset is massive, the system uses "Materialized Views" to speed up the loading of charts.
PRIORITY:	Medium (Important for business growth).
WHEN AVAILABLE:	Final increment / Post-launch.
FREQUENCY OF USE:	Daily/Weekly by management.
CHANNEL TO ACTOR:	Admin Web Interface (React/Charts.js).
SECONDARY ACTOR:	Analytics Engine (Internal Service).
CHANNELS TO SECONDARY ACTORS:	Read-only SQL Queries.
OPEN ISSUES:	1) Should we allow exporting these reports to PDF or Excel? 2) Should we include a "Predictive" feature to guess future price drops?

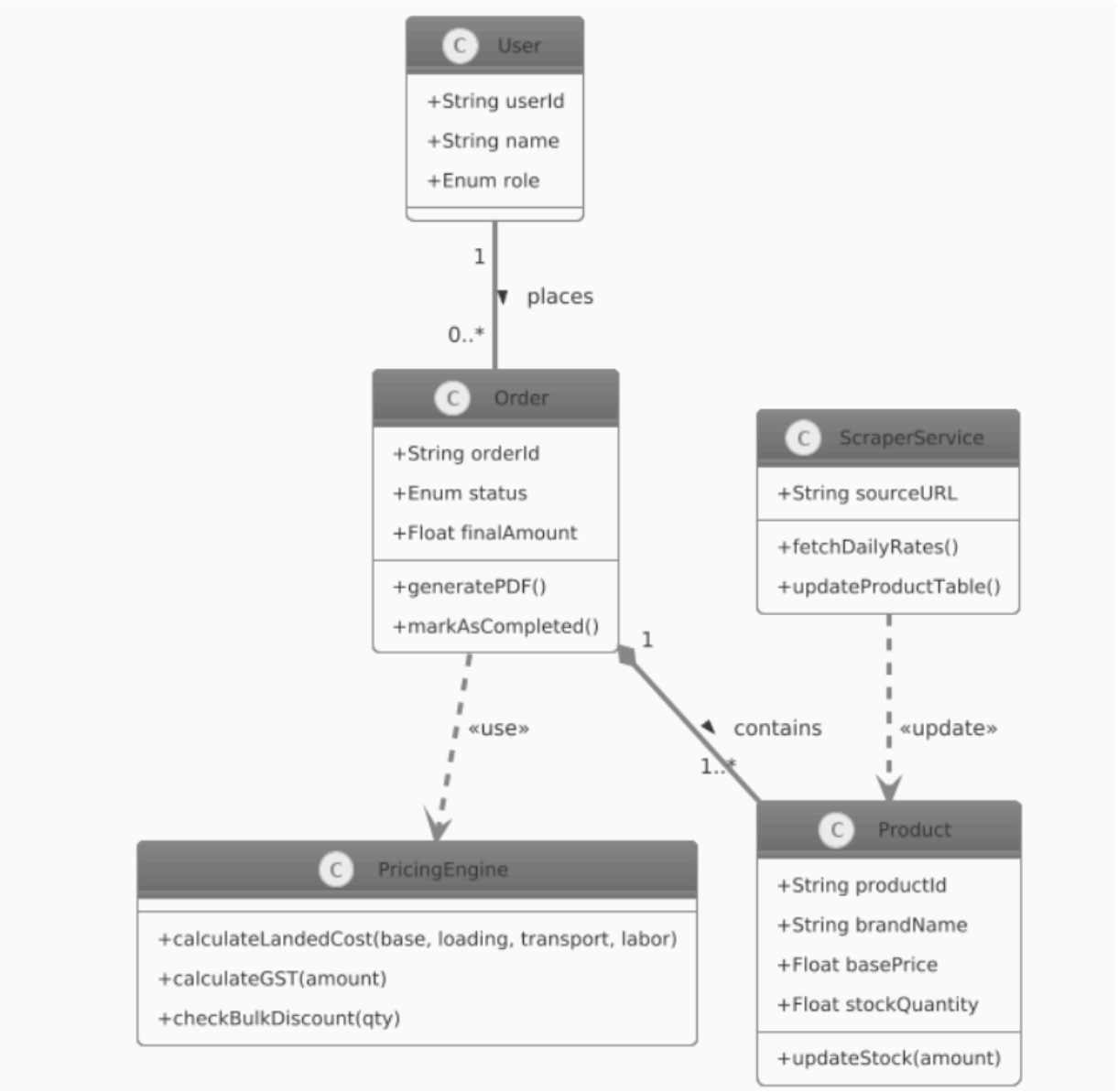
2. Architecture Diagram



The System Architecture Diagram illustrates the overall structure of the Building Materials Management System (BMMS) using a layered architecture approach. The **Presentation Layer** consists of the Customer Storefront and Admin Dashboard built using React and Tailwind, which allow users to browse products, place orders, and manage stock and pricing. These interfaces communicate with the **Application Layer (Node.js)** through an API Gateway that processes requests.

The **Business Logic Modules** include the Scraper Service, Pricing Engine, and Inventory Manager. The Scraper Service retrieves market price data from external websites such as BankBazaar and IndiaMart, while the Pricing Engine calculates the final product cost including additional charges. The Inventory Manager automatically updates stock levels when orders are placed. All system data, including products, prices, orders, and inventory logs, are stored in a **PostgreSQL database**, ensuring reliable data management and system operation

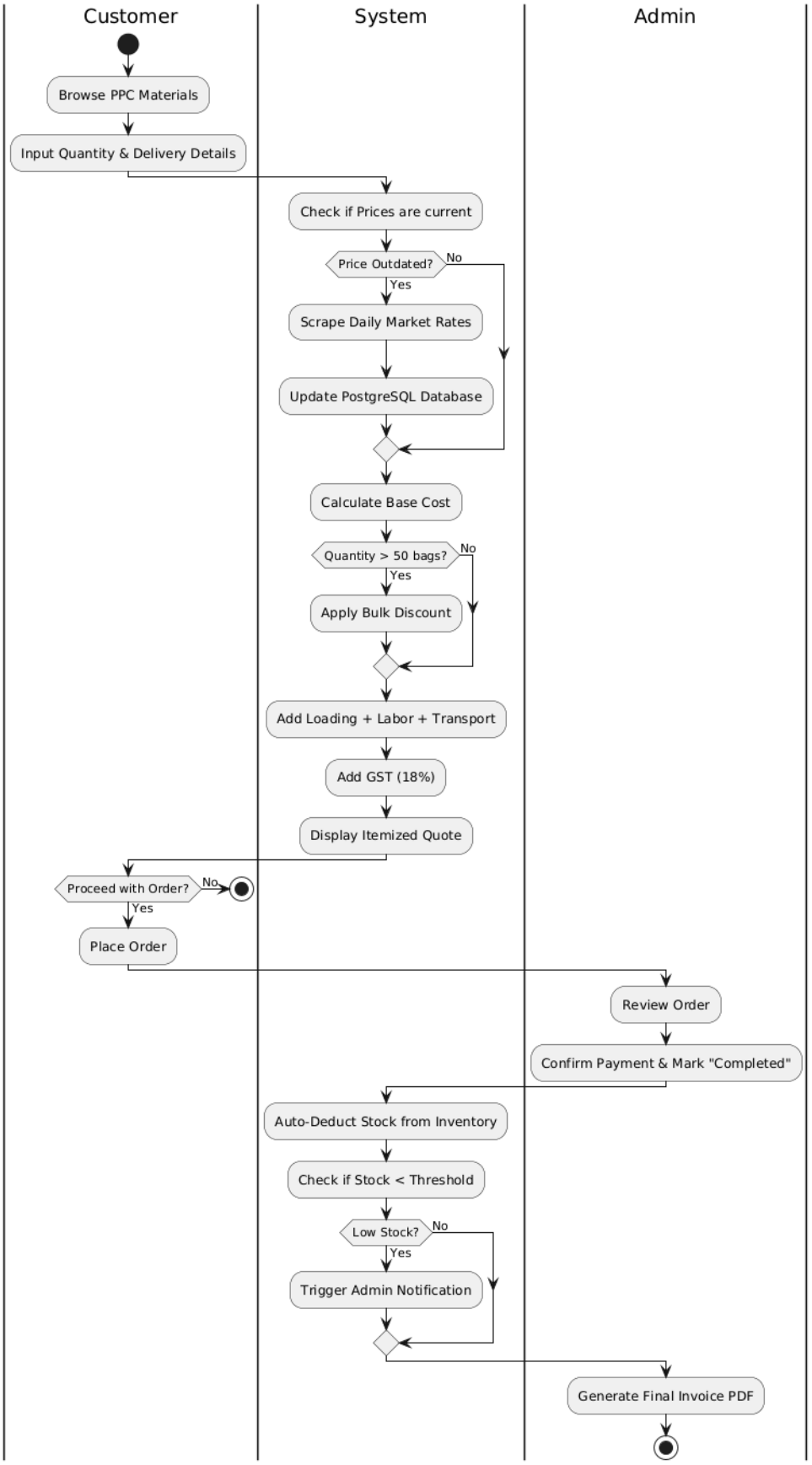
3. Class Diagram



The Class Diagram represents the static structure of the Building Materials Management System (BMMS) by showing the main classes, their attributes, methods, and relationships. The primary classes in the system include **User, Order, Product, PricingEngine, and ScraperService**. The **User** class represents system users who can place orders, while the **Order** class stores order details such as order ID, status, and final amount.

The **Product** class maintains product information including brand name, base price, and stock quantity, and provides methods to update inventory. The **PricingEngine** class is responsible for calculating the final product cost by adding transportation, loading, labor charges, GST, and applying bulk discounts when applicable. The **ScraperService** retrieves daily market prices from external sources and updates the product database. The relationships between these classes illustrate how users place orders, orders contain products, and system services manage pricing and product updates.

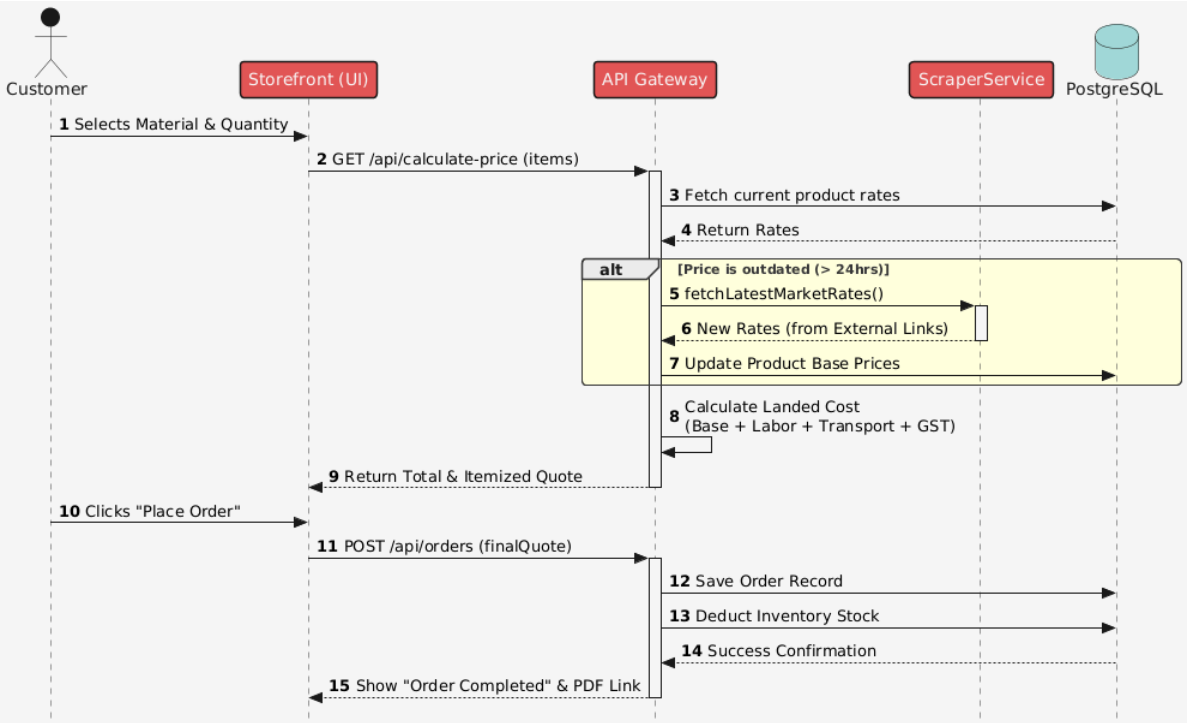
4. Activity Diagram



The Activity Diagram illustrates the workflow of the Building Materials Management System during the process of browsing materials, calculating prices, and placing an order. The process begins when the **customer browses building materials and enters the required quantity and delivery details**. The system then checks whether the product prices are up to date; if not, it retrieves the latest market rates through web scraping and updates the database.

After verifying prices, the system **calculates the base cost of the materials**, applies bulk discounts if the quantity exceeds a specified limit, and adds additional charges such as loading, labor, transportation, and GST. The system then displays an itemized quotation to the customer. If the customer proceeds with the order, the admin reviews and confirms the payment. Finally, the system automatically **updates inventory, checks for low stock alerts, and generates the final invoice PDF**, completing the transaction process.

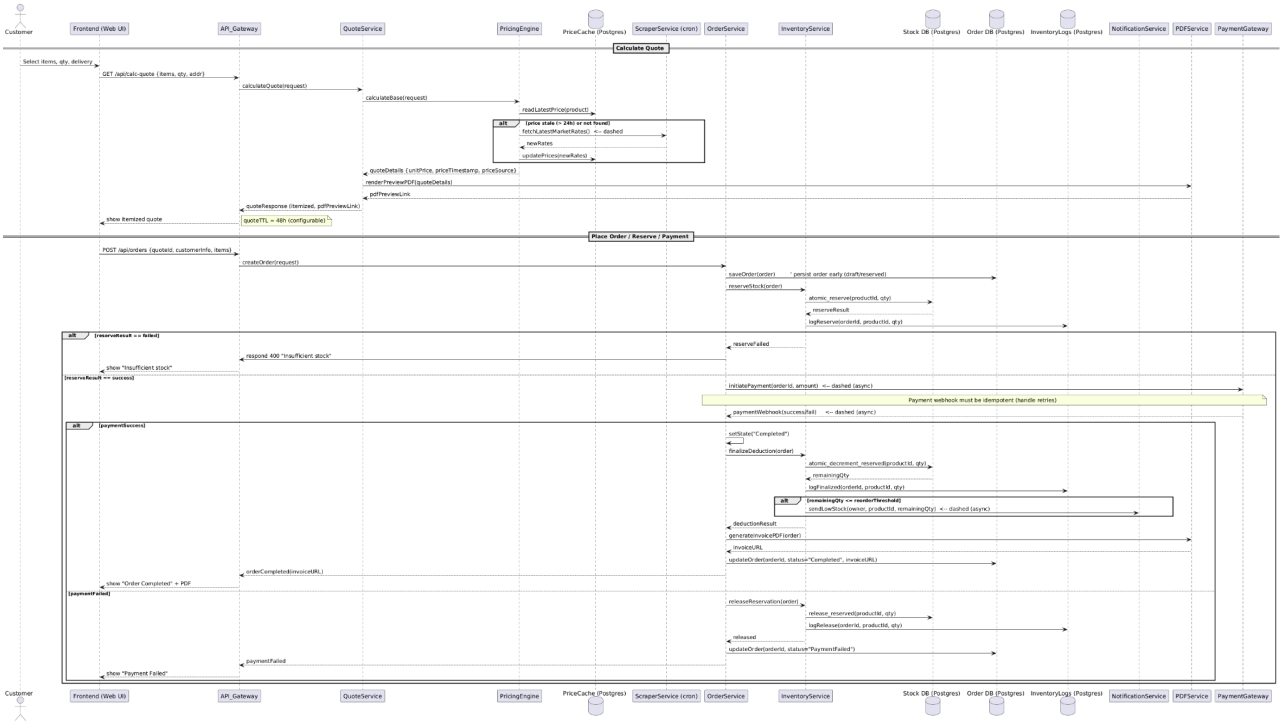
5. Sequence Diagram



The Sequence Diagram illustrates the step-by-step interaction between different components of the Building Materials Management System during the process of calculating price and placing an order. The main participants involved are **Customer, Storefront (UI), API Gateway, Scraper Service, and PostgreSQL Database**.

The process begins when the **customer selects the required material and quantity** through the Storefront interface. The Storefront sends a request to the API Gateway to calculate the price of the selected items. The API Gateway then retrieves the current product rates from the database.

After obtaining the updated rates, the system **calculates the landed cost**, which includes base price, labor charges, transportation cost, and GST. The API Gateway then returns the total itemized quotation to the Storefront. When the customer confirms the order by clicking “Place Order,” the order details are sent to the API Gateway, which stores the order in the database, deducts the inventory stock, and sends a success confirmation. Finally, the Storefront displays the message **“Order Completed” along with a downloadable PDF invoice link**.



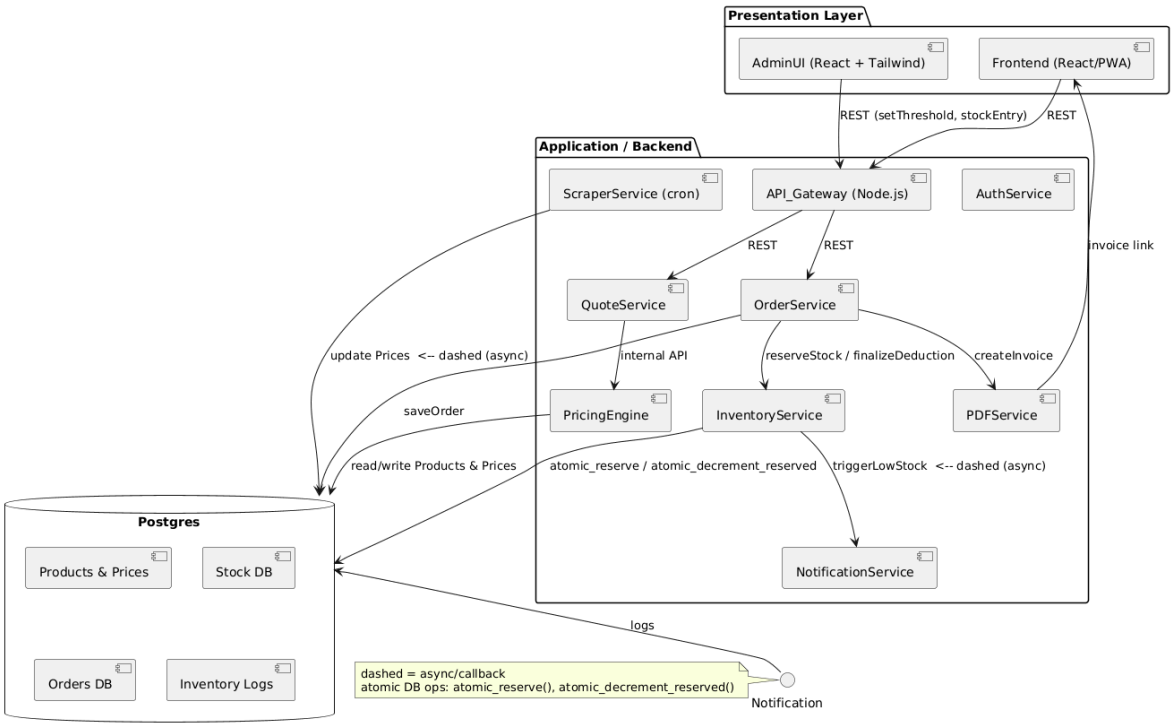
The Communication Diagram illustrates how different components of the Building Materials Management System interact with each other by exchanging messages during the process of generating a quotation and completing an order. The primary participants involved in this communication include **Customer, Frontend (Web UI), API Gateway, Quote Service, Pricing Engine, Scraper Service, Order Service, Inventory Service, Payment Gateway, and various PostgreSQL databases**.

The process begins when the **customer selects materials, quantity, and delivery details through the frontend interface**. The request is sent to the API Gateway, which forwards it to the Quote Service and Pricing Engine to calculate the base price. If the stored product prices are outdated or unavailable, the Scraper Service retrieves the latest market rates and updates the price cache database.

Once the quote is generated, the frontend displays an itemized quotation to the customer. When the customer proceeds with the order, the Order Service records the order details and the Inventory Service reserves the required stock in the database. The system then initiates payment through the Payment Gateway. If the payment is successful, the system updates the order status, deducts inventory stock, generates a PDF invoice, and sends notifications if stock levels fall below a defined threshold. If the payment fails, the reserved stock is released and the order status is updated accordingly.

This diagram helps visualize **how system components collaborate and communicate to complete the full order and payment workflow efficiently**.

6. Component Diagram



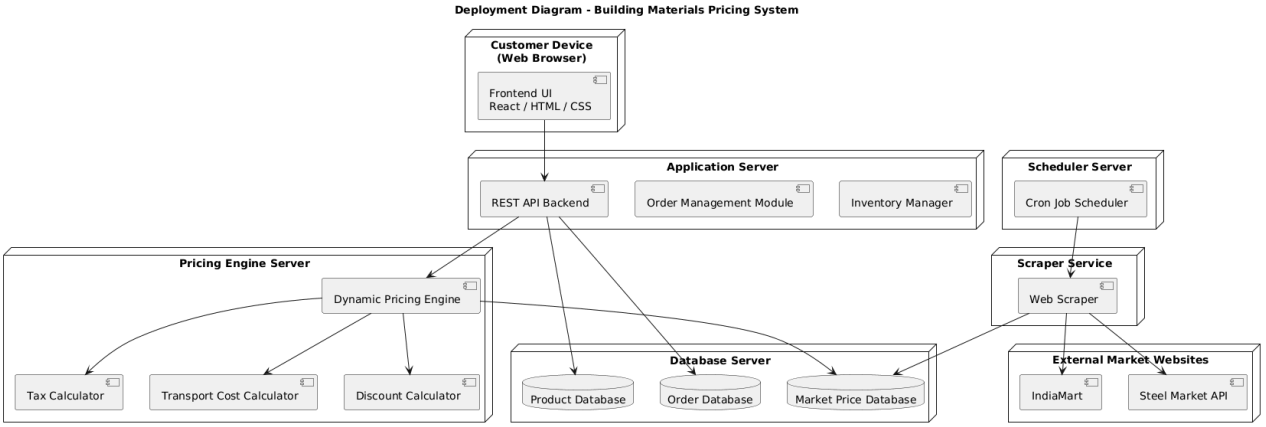
The Component Diagram illustrates the modular structure of the Building Materials Management System and shows how different software components interact with each other. The system is divided into three main layers: **Presentation Layer, Application/Backend Layer, and Database Layer**.

The **Presentation Layer** includes the **Admin UI and Frontend (React/PWA)** interfaces, which allow customers and administrators to interact with the system. These interfaces communicate with the backend through the **API Gateway (Node.js)** using REST APIs. The backend layer contains multiple functional services such as **QuoteService, OrderService, PricingEngine, InventoryService, ScraperService, AuthService, PDFService, and NotificationService**, each responsible for handling specific system operations.

The **ScraperService** periodically collects updated market prices and updates the pricing database. The **QuoteService and PricingEngine** calculate material prices and generate quotations. The **OrderService** manages order creation and processing, while the **InventoryService** handles stock reservation and updates. The **PDFService** generates invoice documents, and the **NotificationService** sends alerts such as low-stock notifications.

All system data is stored in the **PostgreSQL database**, which maintains separate sections for **Products & Prices, Stock Database, Orders Database, and Inventory Logs**. This component-based architecture ensures modularity, scalability, and efficient communication between different system services.

7. Deployment Diagram



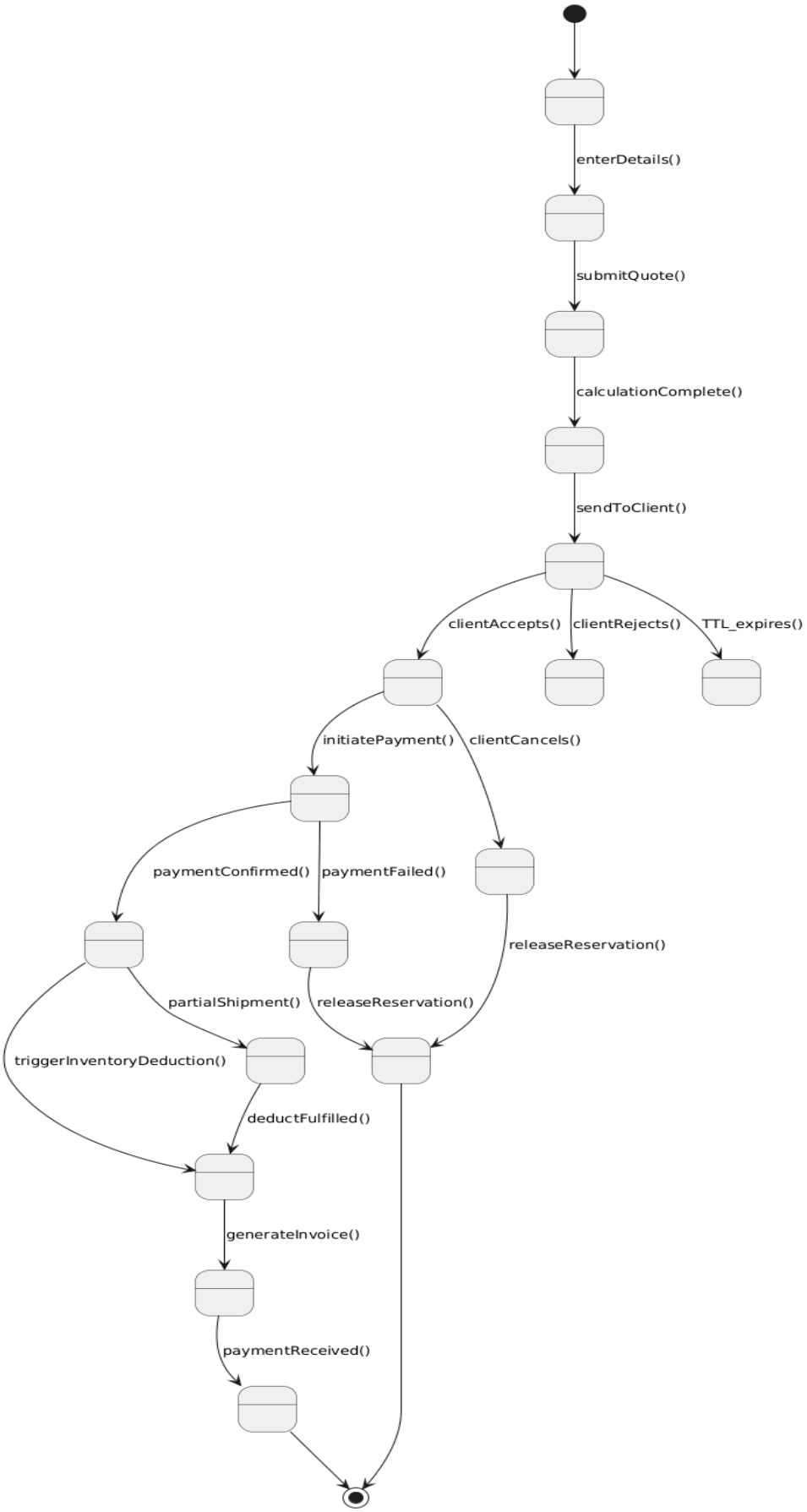
The Deployment Diagram represents the physical deployment of the Building Materials Pricing System across different hardware nodes and servers. It illustrates how software components are distributed and interact within the system environment.

The **Customer Device (Web Browser)** represents the client side where users access the system through a frontend interface developed using **React, HTML, and CSS**. The frontend communicates with the **Application Server** through REST APIs. The Application Server hosts the **REST API Backend**, **Order Management Module**, and **Inventory Manager**, which handle user requests, process orders, and manage stock information.

A separate **Pricing Engine Server** runs the **Dynamic Pricing Engine**, which calculates the final product price by applying tax, transportation cost, and discount calculations. The system also includes a **Scheduler Server** that runs cron jobs responsible for triggering the **Scraper Service** periodically. The Scraper Service collects updated market prices from external sources such as **IndiaMart and Steel Market APIs**.

All system data is stored in the **Database Server**, which maintains multiple databases including the **Product Database**, **Order Database**, and **Market Price Database**. This deployment architecture ensures efficient system performance, separation of services, and scalability by distributing system components across different servers and services.

8. State Machine Diagram



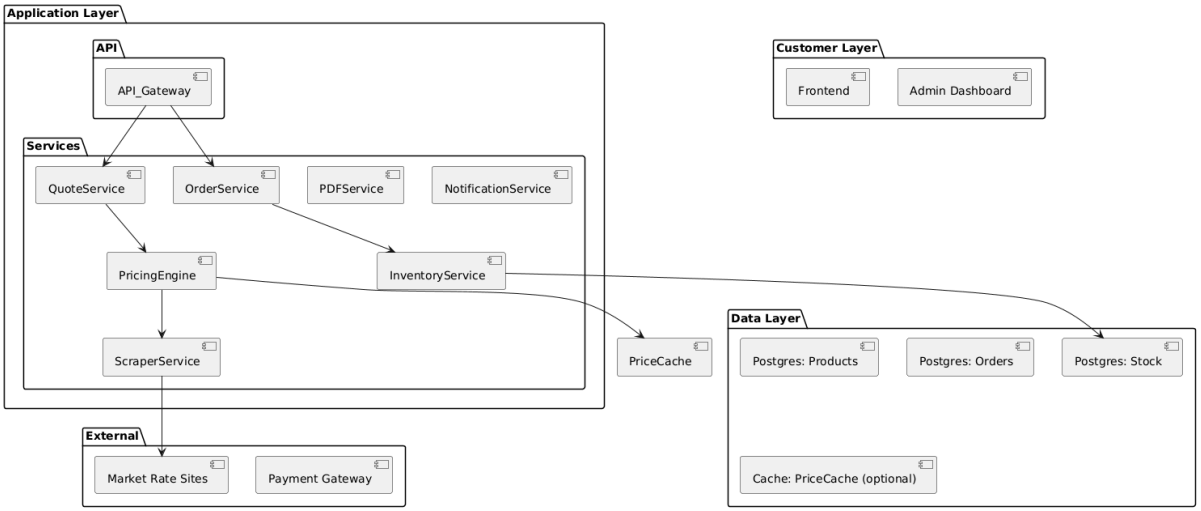
The State Machine Diagram represents the different states and transitions involved in the order and quotation process of the Building Materials Pricing System. It illustrates how the system changes states based on user actions and system events during the lifecycle of a quotation and order.

The process begins when the **customer enters the required material details**, after which the system processes the request and submits the quotation for calculation. Once the price calculation is completed, the system sends the quotation to the client. At this stage, the client can either **accept the quote, reject it, or allow the quote to expire after a specified time limit (TTL)**.

If the client accepts the quotation, the system proceeds to **initiate the payment process**. Depending on the payment outcome, the system transitions to either **payment confirmed** or **payment failed** states. In case of payment confirmation, the system triggers inventory deduction, processes partial shipment if necessary, and generates the invoice. If the payment fails or the client cancels the request, the system releases the reserved inventory.

Finally, once the order is successfully processed and payment is received, the system reaches the **final state**, indicating the completion of the order workflow. This diagram helps in understanding the dynamic behavior of the system and how it handles different conditions and state transitions during order processing.

9. Package Diagram



The Package Diagram represents the high-level organization of the Building Materials Management System by grouping related components into logical packages. This diagram helps in understanding how the system is modularized into different layers and how these layers interact with each other.

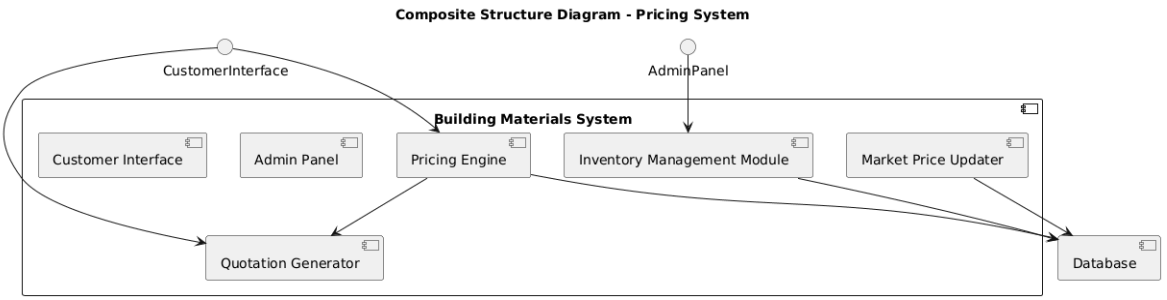
The system is divided into three primary packages: **Customer Layer, Application Layer, and Data Layer**. The **Customer Layer** contains the **Frontend** and **Admin Dashboard**, which provide interfaces for customers and administrators to interact with the system. These interfaces communicate with the backend through the **API Gateway** present in the Application Layer.

The **Application Layer** contains the core services responsible for system functionality. These include **QuoteService, OrderService, PDFService, NotificationService, PricingEngine, InventoryService, and ScraperService**. The API Gateway manages incoming requests and routes them to the appropriate services. The Pricing Engine calculates product prices, while the Scraper Service retrieves updated market rates from external sources. The Order Service processes orders and the Inventory Service manages stock updates.

The **Data Layer** stores and manages system data using **PostgreSQL databases**, which include separate databases for **Products, Orders, and Stock**. An optional **PriceCache** is used to temporarily store recently fetched price data to improve system performance. Additionally, the system interacts with **external services** such as market rate websites for price updates and a payment gateway for transaction processing.

This package-based structure improves system maintainability, scalability, and clear separation of responsibilities among different modules of the system.

10. Composite Structure Diagram



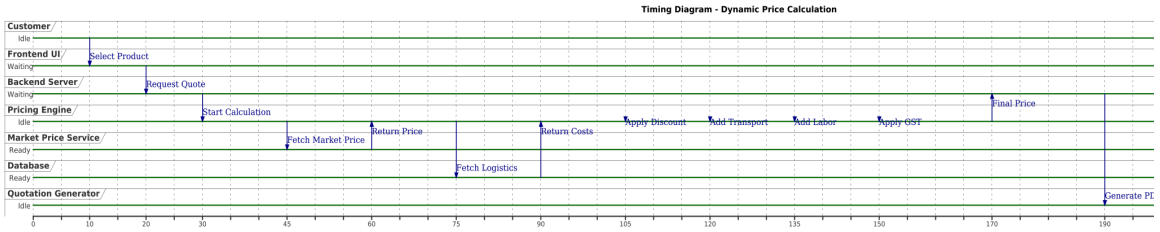
The Composite Structure Diagram illustrates the internal structure of the **Building Materials Pricing System** and shows how different components collaborate within the system to perform various operations. It focuses on how the internal modules interact with each other and with external interfaces.

The system consists of key components such as **Customer Interface, Admin Panel, Pricing Engine, Inventory Management Module, Market Price Updater, Quotation Generator, and Database**. The **Customer Interface** allows users to browse materials, request quotations, and place orders, while the **Admin Panel** enables administrators to manage products, monitor inventory, and oversee system operations.

The **Pricing Engine** processes material prices by calculating base cost, additional charges, and applicable taxes. The **Inventory Management Module** monitors stock levels and updates the inventory whenever orders are placed. The **Market Price Updater** periodically fetches updated material prices from external market sources and stores them in the system database.

The **Quotation Generator** uses the calculated prices to create detailed quotations for customers. All system components interact with the **Database**, which stores information related to products, prices, inventory levels, and order records. This diagram helps demonstrate how the internal parts of the system are structured and how they collaborate to perform the overall pricing and order management functions.

11. Timing Diagram



The Timing Diagram illustrates how different components of the Building Materials Pricing System interact over time during the dynamic price calculation process. It focuses on the sequence of state changes of system components such as **Customer, Frontend UI, Backend Server, Pricing Engine, Market Price Service, Database, and Quotation Generator**.

The process begins when the **customer selects a product** through the Frontend UI. The Frontend sends a **quote request** to the Backend Server, which then triggers the **Pricing Engine** to start the price calculation process. The Pricing Engine retrieves the latest market price from the **Market Price Service**, which accesses the **Database** to obtain updated pricing information.

After receiving the base market price, the system continues the calculation by fetching logistics-related costs from the database. The Pricing Engine then sequentially applies **discounts, transportation charges, labor costs, and GST** to determine the final product price. Once the final price is computed, it is returned to the Backend Server and displayed on the Frontend UI.

Finally, the **Quotation Generator** creates a detailed quotation document in PDF format for the customer. This diagram helps visualize how system components change states and interact with each other over time during the price calculation and quotation generation process.

APPENDIX C: ALGORITHM DESIGN

C.1 Product Price Update Algorithm

• **Problem:** Updating building material prices automatically without manual effort.

• **Solution (Web Scraping Logic):**

1. System runs a **daily scheduled task** to check market prices.
2. Connects to **trusted construction material websites**.
3. Extracts **latest cement and steel prices**.
4. Updates the **price database**.
5. Updated prices are displayed on the **product page**.

C.2 Order Cost Calculation Logic

Triggered when customer places an order:

material_cost = price × quantity

total_cost = material_cost + transport_charge + labor_charge

gst = total_cost × gst_rate

final_amount = total_cost + gst - discount

• System calculates **final cost including GST, transport, and discounts**.
• Generates **invoice for the customer**.

C.3 Inventory Update Algorithm

• **Problem:** Maintaining correct stock levels after orders.

• **Solution:**

1. Customer places order for materials.
2. System checks **available stock**.
3. If stock is available → order is approved.
4. System **reduces stock quantity** from inventory.
5. If stock becomes low → **admin notification is triggered**.

APPENDIX D: IMPLEMENTATION PLAN

D.1 Folder Structure

```
/building-material-system
  /frontend (React / HTML / CSS / JavaScript)
    /pages
    /components
  /backend (Node.js / Express)
    /routes
    /controllers
  /database (MySQL)
  /scripts (Web Scraping for price updates)
  /config
  /server.js
```

D.2 Key API Endpoints

• **POST /api/auth/login**
→ Used for user login and authentication.

• **GET /api/products**
→ Fetch all available building materials and prices.

• **POST /api/orders/create**
→ Create a new order for selected materials.

• **GET /api/inventory/status**
→ Retrieve current stock information.

• **GET /api/admin/reports**
→ Generate sales and inventory reports.

APPENDIX E: TESTING STRATEGY

Test ID	Scenario	Input	Outcome
TC-01	User Login	Valid username & password	Login successful, dashboard displayed
TC-02	Product View	User opens product page	Materials and prices displayed
TC-03	Order Placement	Select product and quantity	Order created successfully
TC-04	Cost Calculation	Quantity + transport charges	Stock < minimum limit
TC-05	Low Stock Alert	Stock < minimum limit	Admin receives low stock notification